



```
In [1]: import os
print(os.listdir())

['.anaconda', '.arduinoIDE', '.conda', '.continuum', '.copilot', '.gitconfig',
'idlerc', '.ipynb_checkpoints', '.ipython', '.jupyter', '.Justread.dat2', '.ke
ras', '.lessht', '.m2', '.matplotlib', '.node_repl_history', '.th-client', '.v
scode', '.wslconfig', 'anaconda3', 'anaconda_projects', 'AppData', 'Application
Data', 'brain_tumor_dataset', 'Contacts', 'Cookies', 'Documents', 'Downloads',
'Favorites', 'IdeaProjects', 'IdeaSnapshots', 'Links', 'Local Settings', 'Mandi
ra - Personal - Shortcut.lnk', 'Microsoft', 'mnist_MandiraNirankari.ipynb', 'Mu
sic', 'My Documents', 'NetHood', 'NTUSER.DAT', 'ntuser.dat.LOG1', 'ntuser.dat.L
OG2', 'NTUSER.DAT{180cb6e6-8059-11ef-ae06-bebb1419005a}.TM.blf', 'NTUSER.DAT{18
0cb6e6-8059-11ef-ae06-bebb1419005a}.TMContainer0000000000000000000001.regtrans-m
s', 'NTUSER.DAT{180cb6e6-8059-11ef-ae06-bebb1419005a}.TMContainer00000000000000
0000002.regtrans-ms', 'ntuser.ini', 'OneDrive', 'Oracle', 'Postman', 'PrintHoo
d', 'projects', 'Recent', 'Saved Games', 'Searches', 'SendTo', 'Start Menu', 's
ubmission.csv', 'Templates', 'Untitled.ipynb', 'Untitled1.ipynb', 'Videos']
```

```
In [2]: os.listdir("brain_tumor_dataset")
```

```
Out[2]: ['.ipynb_checkpoints', 'no', 'yes']
```

```
In [4]: !pip install opencv-python
```

Collecting opencv-python

Downloading opencv_python-4.13.0.92-cp37-abi3-win_amd64.whl.metadata (20 kB)
Requirement already satisfied: numpy>=2 in c:\users\mandi\anaconda3\lib\site-packages (from opencv-python) (2.3.5)

Downloading opencv_python-4.13.0.92-cp37-abi3-win_amd64.whl (40.2 MB)

```
----- 0.0/40.2 MB ? eta -:-:-
----- 0.8/40.2 MB 7.6 MB/s eta 0:00:06
-- ----- 2.6/40.2 MB 7.6 MB/s eta 0:00:05
--- ----- 3.9/40.2 MB 6.8 MB/s eta 0:00:06
----- 5.5/40.2 MB 7.2 MB/s eta 0:00:05
----- 8.9/40.2 MB 9.0 MB/s eta 0:00:04
----- 11.3/40.2 MB 9.5 MB/s eta 0:00:04
----- 15.5/40.2 MB 11.0 MB/s eta 0:00:03
----- 18.4/40.2 MB 11.4 MB/s eta 0:00:02
----- 21.5/40.2 MB 11.9 MB/s eta 0:00:02
----- 24.6/40.2 MB 12.3 MB/s eta 0:00:02
----- 26.2/40.2 MB 11.9 MB/s eta 0:00:02
----- 27.0/40.2 MB 11.5 MB/s eta 0:00:02
----- 28.0/40.2 MB 10.7 MB/s eta 0:00:02
----- 29.1/40.2 MB 10.3 MB/s eta 0:00:02
----- 29.9/40.2 MB 9.9 MB/s eta 0:00:02
----- 31.2/40.2 MB 9.7 MB/s eta 0:00:01
----- 32.2/40.2 MB 9.3 MB/s eta 0:00:01
----- 32.8/40.2 MB 9.0 MB/s eta 0:00:01
----- 33.3/40.2 MB 8.8 MB/s eta 0:00:01
----- 34.1/40.2 MB 8.3 MB/s eta 0:00:01
----- 34.6/40.2 MB 8.0 MB/s eta 0:00:01
----- 35.4/40.2 MB 7.9 MB/s eta 0:00:01
----- 36.4/40.2 MB 7.7 MB/s eta 0:00:01
----- 37.2/40.2 MB 7.6 MB/s eta 0:00:01
----- 37.5/40.2 MB 7.5 MB/s eta 0:00:01
----- 37.7/40.2 MB 7.3 MB/s eta 0:00:01
----- 38.3/40.2 MB 6.9 MB/s eta 0:00:01
----- 38.8/40.2 MB 6.8 MB/s eta 0:00:01
----- 39.6/40.2 MB 6.6 MB/s eta 0:00:01
----- 40.1/40.2 MB 6.6 MB/s eta 0:00:01
----- 40.2/40.2 MB 6.5 MB/s 0:00:06
```

Installing collected packages: opencv-python

Successfully installed opencv-python-4.13.0.92

```
In [5]: import numpy as np
import matplotlib.pyplot as plt
import cv2
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers, models, Model
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_s
import seaborn as sns
```

```
In [6]: datagen = ImageDataGenerator(rescale=1./255, validation_split=0.2)

train_data = datagen.flow_from_directory(
    'brain_tumor_dataset',
```

```

        target_size=(128, 128),
        batch_size=32,
        class_mode='binary',
        subset='training'
    )

val_data = datagen.flow_from_directory(
    'brain_tumor_dataset',
    target_size=(128, 128),
    batch_size=32,
    class_mode='binary',
    subset='validation',
    shuffle=False
)

CLASS_NAMES = {v: k for k, v in train_data.class_indices.items()}
print('Classes:', train_data.class_indices)

```

Found 203 images belonging to 3 classes.

Found 50 images belonging to 3 classes.

Classes: {'.ipynb_checkpoints': 0, 'no': 1, 'yes': 2}

In [7]: *## 5. CNN Model*

```

from tensorflow.keras import Input

inp = Input(shape=(128, 128, 3))
x = layers.Conv2D(32, (3,3), activation='relu', padding='same', name='conv1')(
x = layers.MaxPooling2D((2,2))(x)
x = layers.Conv2D(64, (3,3), activation='relu', padding='same', name='conv2')(
x = layers.MaxPooling2D((2,2))(x)
x = layers.Conv2D(128, (3,3), activation='relu', padding='same', name='conv3_1
x = layers.MaxPooling2D((2,2))(x)
x = layers.Flatten()(x)
x = layers.Dense(128, activation='relu')(x)
x = layers.Dropout(0.5)(x)
out = layers.Dense(1, activation='sigmoid')(x)

model = Model(inp, out)
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy
model.summary()

```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 128, 128, 3)	0
conv1 (Conv2D)	(None, 128, 128, 32)	896
max_pooling2d (MaxPooling2D)	(None, 64, 64, 32)	0
conv2 (Conv2D)	(None, 64, 64, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 64)	0
conv3_last (Conv2D)	(None, 32, 32, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 128)	0
flatten (Flatten)	(None, 32768)	0
dense (Dense)	(None, 128)	4,194,432
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

Total params: 4,287,809 (16.36 MB)

Trainable params: 4,287,809 (16.36 MB)

Non-trainable params: 0 (0.00 B)

```
In [8]: history = model.fit(
        train_data,
        epochs=10,
        validation_data=val_data
    )
```

Epoch 1/10
 7/7  11s 925ms/step - accuracy: 0.3695 - loss: -43.4028 - val_accuracy: 0.3800 - val_loss: -259.8745

Epoch 2/10
 7/7  3s 402ms/step - accuracy: 0.3892 - loss: -1125.8407 - val_accuracy: 0.3800 - val_loss: -4157.1147

Epoch 3/10
 7/7  3s 348ms/step - accuracy: 0.3892 - loss: -9607.6006 - val_accuracy: 0.3800 - val_loss: -28924.8809

Epoch 4/10
 7/7  3s 361ms/step - accuracy: 0.3892 - loss: -49138.5781 - val_accuracy: 0.3800 - val_loss: -129421.9062

Epoch 5/10
 7/7  3s 384ms/step - accuracy: 0.3892 - loss: -201987.1094 - val_accuracy: 0.3800 - val_loss: -451123.7188

Epoch 6/10
 7/7  3s 366ms/step - accuracy: 0.3892 - loss: -610598.2500 - val_accuracy: 0.3800 - val_loss: -1308070.5000

Epoch 7/10
 7/7  3s 426ms/step - accuracy: 0.3892 - loss: -1706676.6250 - val_accuracy: 0.3800 - val_loss: -3280835.7500

Epoch 8/10
 7/7  3s 384ms/step - accuracy: 0.3892 - loss: -3727800.7500 - val_accuracy: 0.3800 - val_loss: -7509832.5000

Epoch 9/10
 7/7  3s 350ms/step - accuracy: 0.3892 - loss: -8171885.0000 - val_accuracy: 0.3800 - val_loss: -15733985.0000

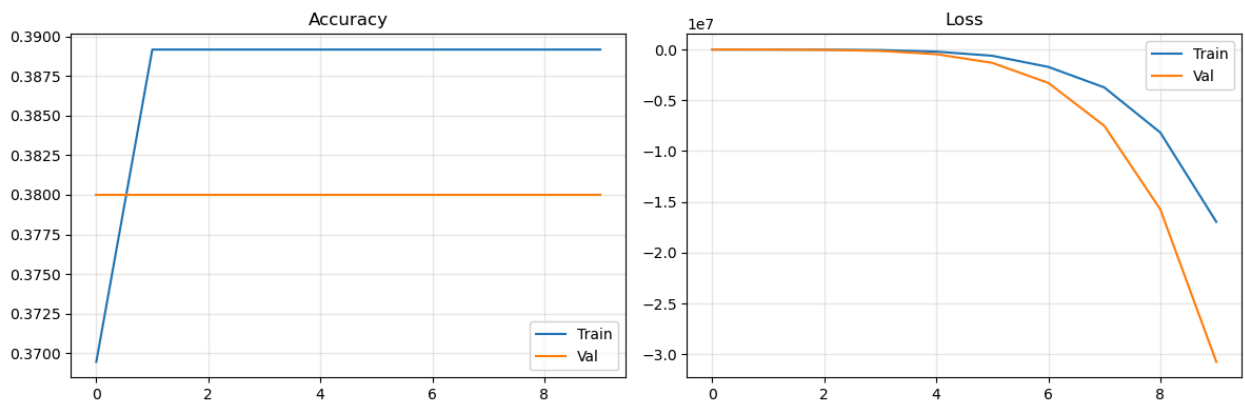
Epoch 10/10
 7/7  3s 370ms/step - accuracy: 0.3892 - loss: -16966482.0000 - val_accuracy: 0.3800 - val_loss: -30754018.0000

```
In [9]: plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Val')
plt.title('Accuracy'); plt.legend(); plt.grid(alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train')
plt.plot(history.history['val_loss'], label='Val')
plt.title('Loss'); plt.legend(); plt.grid(alpha=0.3)

plt.tight_layout()
plt.show()
```



```
In [13]: val_data.reset()
y_true = val_data.classes
y_prob = model.predict(val_data, verbose=0).ravel()
y_pred = (y_prob >= 0.5).astype(int)

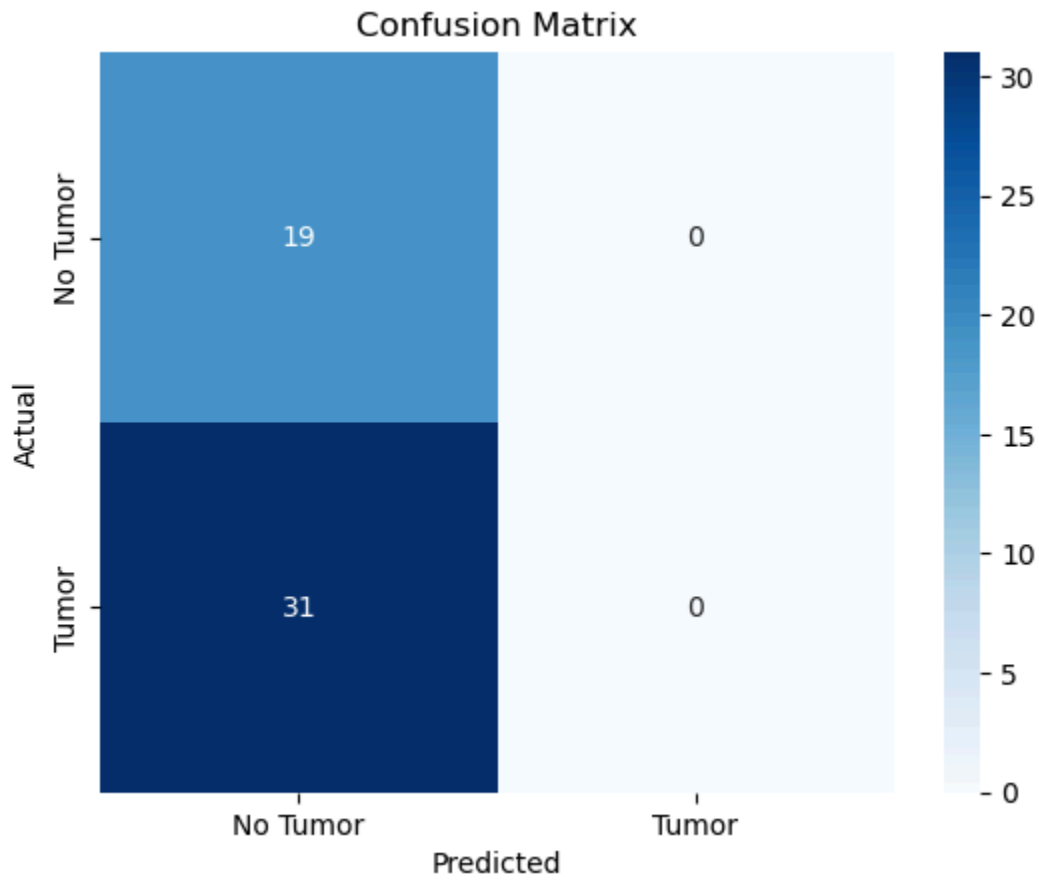
# Validation Accuracy
loss, acc = model.evaluate(val_data, verbose=0)
print(f'Validation Accuracy : {acc:.4f}')
print(f'ROC-AUC           : {roc_auc_score(y_true, y_prob):.4f}')
print()
print(classification_report(y_true, y_pred, target_names=['No Tumor', 'Tumor'])
```

```
Validation Accuracy : 0.3800
ROC-AUC           : 0.5000
```

	precision	recall	f1-score	support
No Tumor	0.38	1.00	0.55	19
Tumor	0.00	0.00	0.00	31
accuracy			0.38	50
macro avg	0.19	0.50	0.28	50
weighted avg	0.14	0.38	0.21	50

```
C:\Users\mandi\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1
731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in l
abels with no predicted samples. Use `zero_division` parameter to control this
behavior.
 warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
C:\Users\mandi\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1
731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in l
abels with no predicted samples. Use `zero_division` parameter to control this
behavior.
 warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
C:\Users\mandi\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1
731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in l
abels with no predicted samples. Use `zero_division` parameter to control this
behavior.
 warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

```
In [14]: # Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['No Tumor', 'Tumor'],
            yticklabels=['No Tumor', 'Tumor'])
plt.title('Confusion Matrix')
plt.ylabel('Actual'); plt.xlabel('Predicted')
plt.show()
```



```
In [15]: for layer_name in ['conv1', 'conv2', 'conv3_last']:
filters = model.get_layer(layer_name).get_weights()[0] # (3,3,in,out)
n = min(16, filters.shape[-1])

# Normalize
f = filters - filters.min()
f = f / (f.max() + 1e-8)

plt.figure(figsize=(10, 2.5))
plt.suptitle(f'Filters - {layer_name}', fontweight='bold')
for i in range(n):
    plt.subplot(2, 8, i+1)
    plt.imshow(f[:, :, :, i].mean(axis=-1), cmap='viridis')
    plt.axis('off')
plt.tight_layout()
plt.show()
```

Filters — conv1



Filters — conv2



Filters — conv3_last



```
In [16]: def get_gradcam(model, img_array, layer_name='conv3_last'):
grad_model = Model(
    inputs=model.input,
    outputs=[model.get_layer(layer_name).output, model.output]
)
with tf.GradientTape() as tape:
    conv_out, preds = grad_model(img_array)
    loss = preds[:, 0]

    grads = tape.gradient(loss, conv_out)
    pooled = tf.reduce_mean(grads, axis=(0,1,2))
    heatmap = tf.nn.relu(conv_out[0] @ pooled[..., tf.newaxis])
    heatmap = heatmap.numpy().squeeze()
    heatmap = heatmap / (heatmap.max() + 1e-8)
    return heatmap

print('Grad-CAM function ready')
```

Grad-CAM function ready

```
In [17]: val_data.reset()
imgs, labels = next(val_data)

plt.figure(figsize=(15, 10))
```



```

for i in range(6):
    img = imgs[i]
    inp = np.expand_dims(img, 0)
    prob = model.predict(inp, verbose=0)[0][0]
    pred = 'Tumor' if prob >= 0.5 else 'No Tumor'
    true = CLASS_NAMES[int(labels[i])]

    heatmap = get_gradcam(model, inp)
    heatmap_resized = cv2.resize(heatmap, (128, 128))
    heatmap_color = cv2.applyColorMap(
        np.uint8(255 * heatmap_resized), cv2.COLORMAP_JET
    )
    heatmap_color = cv2.cvtColor(heatmap_color, cv2.COLOR_BGR2RGB)
    overlay = np.uint8(0.6 * np.uint8(img*255) + 0.4 * heatmap_color)

    # Original
    plt.subplot(6, 3, i*3+1)
    plt.imshow(img)
    plt.title(f'True: {true} | Pred: {pred} ({prob:.2f})', fontsize=8)
    plt.axis('off')

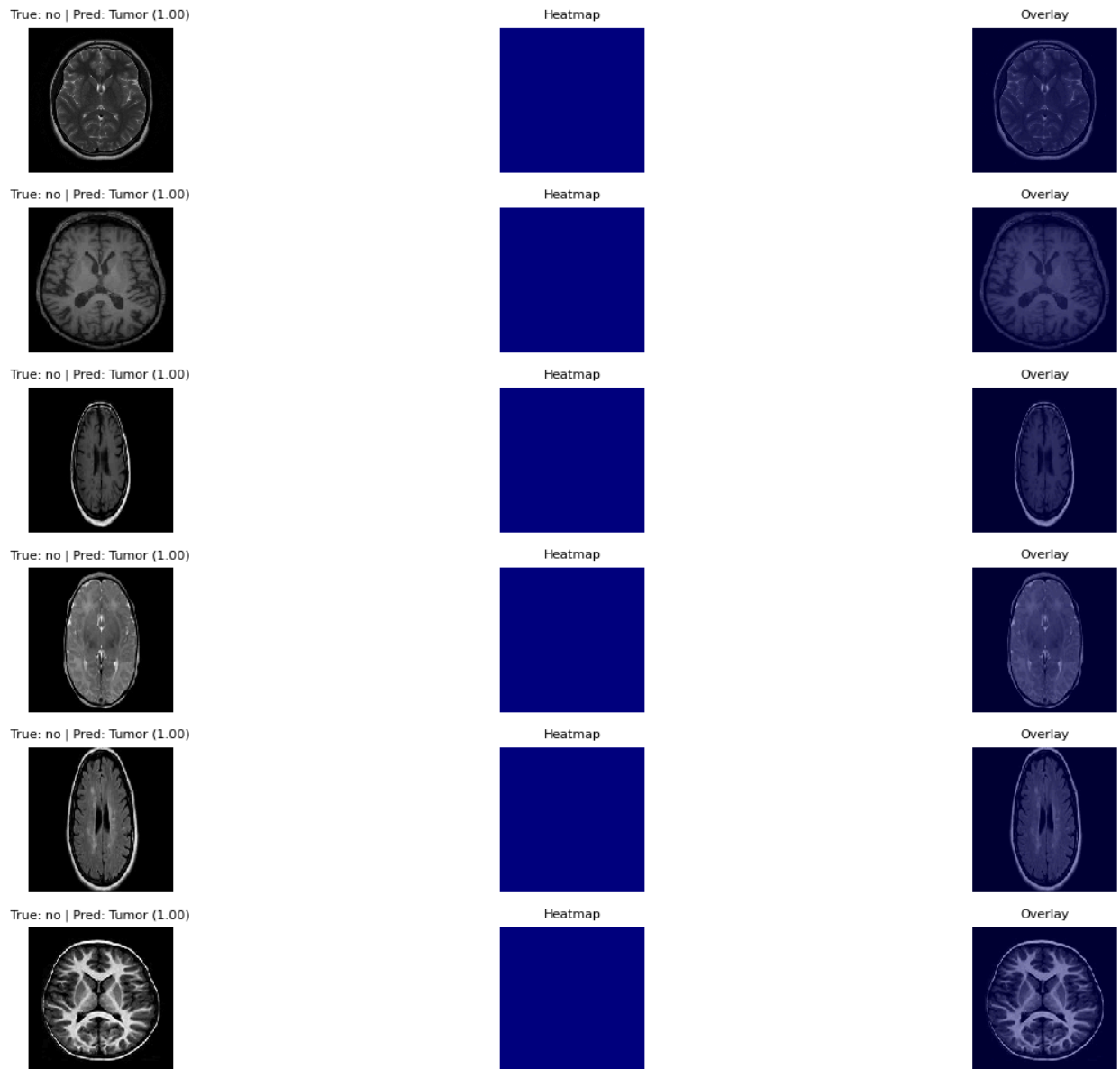
    # Heatmap
    plt.subplot(6, 3, i*3+2)
    plt.imshow(heatmap_color)
    plt.title('Heatmap', fontsize=8)
    plt.axis('off')

    # Overlay
    plt.subplot(6, 3, i*3+3)
    plt.imshow(overlay)
    plt.title('Overlay', fontsize=8)
    plt.axis('off')

plt.suptitle('Grad-CAM: Original | Heatmap | Overlay', fontweight='bold', y=1.05)
plt.tight_layout()
plt.show()

```

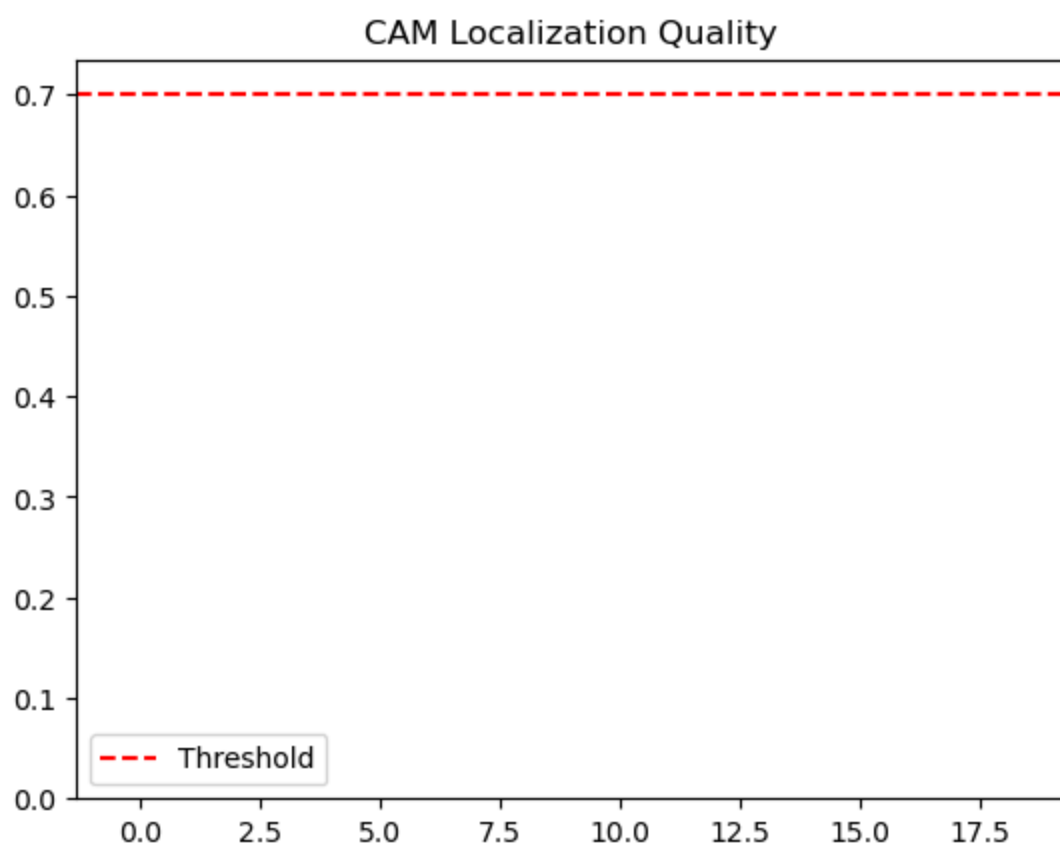
Grad-CAM: Original | Heatmap | Overlay



```
In [18]: ## 11. Localization Quality
val_data.reset()
imgs, labels = next(val_data)

scores = []
for img, lbl in zip(imgs, labels):
    inp = np.expand_dims(img, 0)
    prob = model.predict(inp, verbose=0)[0][0]
    if int(lbl) == 1 and prob >= 0.5:
        h = get_gradcam(model, inp).ravel()
        scores.append(np.sort(h)[::-1][:len(h)//2].sum() / (h.sum() + 1e-8))

plt.bar(range(len(scores)), scores, color='steelblue')
plt.axhline(0.7, color='red', linestyle='--', label='Threshold')
plt.title('CAM Localization Quality'); plt.legend(); plt.show()
```



In []: